

3.2.5 Наївний Баєсів класифікатор (Naïve Bayes) та NLP

У цьому розділі розглядаються основи обробки природної мови (Natural Language Processing, NLP) та аналіз тональності тексту. Sentiment Analysis використовується для автоматичного визначення емоційного забарвлення тексту: позитивного, негативного або нейтрального.

Перед початком роботи необхідно підключити бібліотеки:

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix
```

- **TfidfVectorizer** – перетворює текст у числові ознаки за допомогою TF-IDF (Term Frequency – Inverse Document Frequency). Модель машинного навчання не може працювати з текстом напряму, тому слова переводяться у вектори чисел;
- **MultinomialNB** – реалізація наївного байєсівського класифікатора (Naive Bayes), який часто використовується для задач класифікації тексту;
- **Pipeline** – дозволяє об'єднати кілька етапів обробки даних (наприклад vectorizer + classifier) у єдину модель;

Обробка тексту та виділення ознак

Моделі машинного навчання не можуть працювати з текстом напряму. Перед навчанням текст необхідно перетворити у числовий формат, який можна аналізувати математично.

```
def load_data(pos_path, neg_path):
    with open(pos_path, 'r', encoding='utf-8', errors='ignore') as f:
        pos_text = f.read().splitlines()
    with open(neg_path, 'r', encoding='utf-8', errors='ignore') as f:
        neg_text = f.read().splitlines()

    X = pos_text + neg_text
    y = [1] * len(pos_text) + [0] * len(neg_text)
    return train_test_split(X, y, test_size=0.2, random_state=42)

X_train, X_test, y_train, y_test = load_data('rt-polarity.pos', 'rt-polarity.neg')
print(f"Дані завантажено. Тренувальна вибірка: {len(X_train)} відгуків")
```

У цьому прикладі: `rt-polarity.pos` - містить позитивні відгуки; `rt-polarity.neg` - негативні відгуки. Для читання файлів використовується метод `open()`, тут: `encoding='utf-8'` - підтримка Unicode символів; `errors='ignore'` - ігнорування проблемних символів.

Після цього: `f.read().splitlines()` зчитує увесь текст та розбиває його на окремі рядки (окремі відгуки).

Після завантаження створюються: `X` - список текстів (об'єднує позитивні та негативні відгуки); `y` - створює мітки класів (1 - позитивні, 0 - негативні).

Перш ніж модель почне вчитися, текст проходить через кілька етапів обробки:

- **Токенізація (Tokenization)** - це розбиття тексту на окремі слова (токени). Вони стають базовими елементами для аналізу тексту.
- **Очищення (Preprocessing)** - виконується перед перетворенням тексту в числові ознаки: переводить текст у нижній регістр, видаляє пунктуацію стоп-слова (stop-words). Це допомагає зменшити шум, покращити якість моделі та зменшити кількість непотрібних ознак.
- **Стоп-слова (stop-words)** - це слова, що часто зустрічаються, але майже не несуть емоційного змісту («the», «and», «is», «a»).

Методи представлення тексту:

- **Bag of Words** - текст розглядається як набір слів, граматики та порядок слів ігноруються, враховується лише кількість появи слова. Такий підхід дозволяє перетворити текст у числовий вектор.
- **Розумне зваження TF-IDF (Term Frequency - Inverse Document Frequency)** - покращена версія Bag of Words, зменшує вагу дуже частих слів та збільшує вагу унікальних і важливих.

TF-IDF обчислюється:

$$TFIDF(t, d) = TF(t, d) \cdot \log \frac{N}{df(t)}$$

де:

- $TF(t, d)$ - частота слова у документі,
- $df(t)$ - кількість документів, де зустрічається слово,
- N - загальна кількість документів.

Саме TF-IDF найчастіше використовується у класичних NLP-моделях.

Класифікація тексту (Naïve Bayes)

Після перетворення тексту у числові ознаки можна переходити до навчання моделі класифікації. Одним із найпопулярніших алгоритмів для аналізу тексту є Наївний Байєс (Naïve Bayes).

```
text_clf = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', min_df=5, ngram_range=(1, 2))),
    ('clf', MultinomialNB())
])

text_clf.fit(X_train, y_train)
```

У цьому прикладі використовується Pipeline - спеціальний механізм sklearn, який дозволяє об'єднати кілька етапів обробки даних у єдиний процес. Тут pipeline складається з двох етапів: **TfidfVectorizer** - перетворення тексту у TF-IDF ознаки, **MultinomialNB** — класифікація тексту.

Аргументи TfidfVectorizer: **stop_words='english'**, - автоматично видаляє англійські стоп-слова, **min_df=5** - слово враховується тільки якщо воно зустрічається в 5 документах і більше, **ngram_range=(1, 2)** - модель аналізує окремі слова та пари слів, це допомагає їй краще зрозуміти контекст

Для класифікації використовується MultinomialNB() - це використання теореми Байєса у коді. У задачі аналізу тексту модель оцінює наскільки ймовірно, що текст позитивний, якщо в ньому зустрічаються певні слова. Тобто слово «excellent» збільшує ймовірність позитивного класу, слово «terrible» збільшує ймовірність негативного класу.

Алгоритм називається «наївним» через свій підхід до задачі. Модель вважає появу слово «good» ознакою, що клас ймовірно позитивний. Вона не враховує сарказм чи ситуацію того що людина просто привіталась (добрий день). Але для текстів результати часто виявляються хорошими, тому цей алгоритм ще активно використовується.

Оцінка якості моделі в NLP

Після навчання моделі необхідно перевірити, наскільки добре вона визначає тональність тексту на нових даних.

```

predictions = text_clf.predict(X_test)

print("\n--- Звіт про класифікацію ---")
print(classification_report(y_test, predictions, target_names=['Negative', 'Positive']))

sample_reviews = [
    "This movie was an absolute masterpiece with great acting!",
    "Waste of time. The plot was boring and predictable.",
    "Not bad, but I've seen better"
]

sample_preds = text_clf.predict(sample_reviews)
for review, pred in zip(sample_reviews, sample_preds):
    sentiment = "Positive" if pred == 1 else "Negative"
    print(f"Review: {review[:50]}... -> {sentiment}")

```

Метод: `predict()` використовується для прогнозування класів, тут: `X_test` - тестові відгуки, `predictions` - прогнозовані мітки класів, `target_names=['Negative', 'Positive']` - замінює числа 0 та 1 на зрозумілі назви класів.

Модель можна використовувати не лише для тестового набору, а й для довільних речень (оголомити змінну `sample_reviews`, після чого визначити її тональність `text_clf.predict(sample_reviews)`)

Також можна подивитись, які слова модель вважає найбільш позитивними (функція `show_top_words`).

```

def show_top_words(pipeline, n=10):
    vectorizer = pipeline.named_steps['tfidf']
    clf = pipeline.named_steps['clf']
    feature_names = vectorizer.get_feature_names_out()

    sorted_indices = np.argsort(clf.feature_log_prob_[1])[-n:]
    print(f"\nТоп-{n} слів для позитивного класу:")
    print([feature_names[i] for i in sorted_indices])

show_top_words(text_clf)

```

Pipeline зберігає всі етапи моделі, а через `named_steps` можна отримати доступ до зваження тексту ('tfidf') та класифікатора ('clf'). Команда `np.argsort()` сортує слова за важливістю, тут: `feature_log_prob_` містить логарифми ймовірностей для кожного класу (тобто чим більше значення тим більше слово асоціюється з класом).